

Week 03 — Tool Use

Lecture notes • 45-minute concept briefing

These notes follow the approved 42-slide briefing. Chapter and section numbers match the slides; the slide references identify the corresponding pages. Definitions, prompts, interpretations, and explanatory notes are presented together.

Teaching examples, forecasts, inventory records, and generated responses are illustrative unless explicitly identified otherwise. They are not new measurements of model behavior.

Contents

1. Tools and the tool-use cycle
2. From tool descriptions to tool calls
3. Using tool results in an answer
4. Tool sequences and action outcomes
5. Training models to use tools
6. Evaluating tool use

Introduction

Week 2 recall: reasoning from the supplied input

Slide 2.

Prompting changes how a model uses the information in its input.

Few-shot prompting supplies examples. Chain-of-thought prompting elicits intermediate steps. Self-consistency selects the most frequent answer across sampled solutions.

These methods do not retrieve a new weather forecast. A forecast must be supplied or obtained through an external operation.

The methods can use evidence already supplied in context; the claim concerns acquisition of new observations.

Sources: [Wei et al. \(2022\)](#) · CoT; [Wang et al. \(2022/2023\)](#) · Self-consistency.

Missing facts in a reasoning prompt

Slide 3.

A model needs a weather forecast to give a supported recommendation about tomorrow's weather.

EXAMPLE · A PROMPT WITHOUT A FORECAST:

User: Will I need an umbrella in Seoul tomorrow?

Prompt: Explain your reasoning before giving advice.

Missing information: tomorrow's forecast for Seoul.

A reasoning prompt can organize an explanation. A forecast lookup provides the missing fact on which the advice depends.

Sources: [Ng](#) · [Agentic AI, Module 3](#).

Lecture structure: from a question to a verified outcome

Slide 4.

01 Tools: external operations and the tool-use cycle. (5 min)

02 Calls: tool descriptions, request generation, and execution. (10.5 min)

03 Answers: returning tool results and using them as evidence. (7.5 min)

04 Sequences: using one result in a later action. (5 min)

05 Training: learning tool use in Toolformer and ToolLLM. (11 min)

06 Evaluation: checking requests, evidence, and outcomes. (2.5 min)

Introduction: 3.5 min · Total: 45 min

The chapters follow the information passed among the user, model, application, and tools.

Chapters 1–4 explain tool use during inference. Chapter 5 explains how training shapes request

generation, and Chapter 6 evaluates requests, evidence, and outcomes. Implementation details are provided separately for the later lab.

1. Tools and the tool-use cycle

Slide 5.

1.1 Motivation: tasks beyond text generation

Slide 6.

Generating a sentence cannot retrieve a forecast or reserve an item.

An umbrella recommendation needs weather data. A reservation needs an operation in an inventory system.

Purpose: define a tool and explain how the model's request leads to an external operation.

Sources: [Ng · Agentic AI, Module 3](#); [Schick et al. \(2023\)](#) · [Toolformer](#).

1.2 Tools as external operations

Slide 7.

A tool is an operation outside the language model that software can execute to obtain information or perform an action.

EXAMPLES · TOOL INPUTS AND RESULTS:

Weather lookup: city and day → forecast.

Calculator: arithmetic expression → computed value.

Reservation: item and quantity → reservation outcome.

The application is the software that connects the model to these operations. It executes the requested tool and returns the result.

This definition includes local and remote operations. A tool need not be a web service, and an API is one way of exposing a capability. The categories describe intended effects rather than mutually exclusive implementation types.

Sources: [Ng · Agentic AI, Module 3](#); [Anthropic · Building effective agents](#).

1.3 The tool-use cycle: model and application

Slide 8.

A tool call requests an operation with specific inputs. The application executes it and returns the result, called an observation.

The returned result becomes part of the next model input. The model then generates an answer or another tool call.

Figure description: Application supplies task and tool definitions → model generates a request → application executes the tool → application adds the result to the next input → model generates an answer or another request.

1. Specify: the application supplies the task and descriptions of available tools.

2. Request: the model generates an operation and the input values it needs.
3. Execute: the application checks whether the request can be performed and invokes the external capability. A rejected request is not executed.
4. Observe: the application returns the result or failure information.
5. Continue: the model uses the expanded context to answer or request another operation. A request alone supplies neither a new observation nor a completed external action.

Sources: [Ng · Agentic AI, Module 3](#); [Anthropic · Building effective agents](#).

1.4 Example: a forecast lookup and an answer

Slide 9.

The model can base its advice on a forecast after the application retrieves and returns it.

EXAMPLE · ILLUSTRATIVE REQUEST, RESULT, AND ANSWER:

User: Will I need an umbrella in Seoul tomorrow?

Model request: retrieve Seoul's forecast for tomorrow.

Application: execute the weather lookup and return its result.

Tool result: "Seoul, tomorrow: 80% probability of rain."

Model answer: Rain is likely; carrying an umbrella is advisable.

The request specifies the lookup. The tool result supplies the evidence for the answer.

Concept check: Does a generated lookup request establish that a forecast was retrieved?

Answer: No. Execution and a returned observation are required; request generation alone provides no forecast.

Sources: [Ng · Agentic AI, Module 3](#); [Anthropic · Building effective agents](#).

2. From tool descriptions to tool calls

Slide 10.

2.1 Motivation: making tools known to the model

Slide 11.

A weather question alone does not tell the model that a Forecast tool is available.

The application must supply the tool's description and required inputs before the model can select it for this task.

Purpose: explain how tool information and the user's question lead to a generated call.

Sources: [Anthropic · Tool definitions and model context](#).

2.2 Providing tool information to the model

Slide 12.

The application includes available tool definitions in the model input.

A tool definition states its name, what it does, the inputs it needs, and what it returns.

A system prompt supplies application instructions. Tool definitions can be written there or provided through a separate tool interface. Both routes make the information available to the model.

The claim concerns the described setup with tools supplied for the current interaction. A description informs the model of an available capability; an executable implementation must also exist outside the model. It is not a universal claim that every model must be given a fresh natural-language description of a tool on every occasion: some tool conventions are learned during training. Anthropic documents that separately supplied tool definitions are used to construct a tool-use system prompt. This supports the distinction between information the model receives and whether the developer manually writes that information in the system message. Do not generalize one provider's exact serialization to every model.

Sources: [Anthropic · Tool definitions and model context](#).

2.3 Example: a prompt that enables a forecast request

Slide 13.

The system prompt describes the tools and how to request them.

EXAMPLE · COMPLETE TEACHING PROMPT:

SYSTEM PROMPT

Forecast: returns weather for a given city and day.

Calculator: evaluates an arithmetic expression.

For weather advice, request Forecast before answering.

Write "Tool request:" followed by the tool name and inputs.

Ask for the city if it is missing.

USER MESSAGE

Will I need an umbrella in Seoul tomorrow?

The application must recognize "Tool request:" and run the named tool.

This is a prompt-based teaching protocol that requests a readable text convention. The application must be configured to recognize that convention and connect the named tools to implementations. The example does not claim that mentioning a function in a system prompt activates a provider's native tool interface. Native tool definitions can be supplied separately and incorporated into model context by the service. The purpose here is to expose the information supplied to the model without requiring knowledge of API fields. All following generated outputs are illustrative, not measured model responses.

Sources: [Anthropic · Tool definitions and model context](#).

2.4 Example: choosing a tool and its inputs

Slide 14.

The tool description identifies the operation; the user's question supplies the city and day.

EXAMPLE · FROM SUPPLIED INFORMATION TO A REQUEST:

Available tool: Forecast returns weather for a city and day.

Instruction: Request Forecast before giving weather advice.

User: Will I need an umbrella in Seoul tomorrow?

Illustrative model output:

Tool request: Forecast; city: Seoul; day: tomorrow

Forecast matches the information needed for the task. Seoul and tomorrow fill its required inputs. The lookup has not yet run.

This mapping explains the relevant input and output at the observable level. It is not a claim that a hidden symbolic matcher or a particular chain of thought was measured inside the model. The previous full prompt supplies the request convention and the condition for requesting a forecast. The next page explains how learned token generation can produce the illustrated output.

Sources: [Anthropic · Tool definitions and model context](#); [Schick et al. \(2023\) · Toolformer](#).

2.5 Tool-call selection through next-token generation

Slide 15.

An autoregressive model predicts each token from its input and earlier output tokens. A model trained for tool use can generate a call through this same process.

MECHANISM · REQUEST GENERATION (SCHEMATIC):

Input: tool descriptions, use instructions, and the umbrella question.

Generated beginning: "Tool request:"

Generated continuation: "Forecast; city: Seoul; day: tomorrow"

Each next token depends on the input and the generated text.

Selecting a call is part of generating the response. Training teaches this behavior; the current prompt guides it. An unnecessary or incorrect call remains possible.

The displayed prefix and continuation are readable fragments, not literal token boundaries or a captured internal trace. Autoregressive generation uses $P(\text{next token} \mid \text{supplied input, generated prefix})$. In this teaching protocol, the model generates "Tool request:" and the request content; the application recognizes that convention and connects it to execution. Training shapes these conditional probabilities, while inference uses the learned parameters.

This account assumes neither a universal keyword trigger nor a fixed self-assessed uncertainty threshold. It does not require a separate explicit tool-choice classifier. Toolformer demonstrates learned call initiation, tool identity, inputs, and subsequent use of results with its own call representation. The teaching convention here is neither Toolformer's literal syntax nor a provider-native wire format.

Sources: [Schick et al. \(2023\) · Toolformer](#); [Brown et al. \(2020\) · Few-shot learning](#).

2.6 Required inputs and missing user information

Slide 16.

Arguments are the input values in a tool call. The model should obtain them from the task or other supplied information.

EXAMPLE · COMPLETE AND INCOMPLETE REQUESTS:

Forecast requires: a city and a day.

User: Retrieve tomorrow's forecast for Seoul.

Available arguments: city = Seoul; day = tomorrow.

User: Retrieve tomorrow's forecast here.

Appropriate response: "Which city should I check?"

Grounding arguments means tying them to available information. A tool description specifies required inputs; it cannot supply a missing user location.

The example assumes no location information is present elsewhere in the context. The preceding system prompt explicitly asks for the city when it is missing. The appropriateness of the response is therefore grounded in both task requirements and the supplied instruction. Actual models can fail to follow that instruction.

Sources: [Schick et al. \(2023\) · Toolformer](#); [Anthropic · Tool definitions and model context](#).

2.7 Execution of the call by the application

Slide 17.

The application reads the generated call, checks its inputs, and runs the corresponding tool.

EXAMPLE · A REQUEST BECOMING AN EXTERNAL OPERATION:

Model output: Tool request: Forecast; city: Seoul; day: tomorrow

Application action: execute Forecast with Seoul and tomorrow.

Tool result: "Seoul, tomorrow: 80% probability of rain."

Function calling provides a structured format for the same request. The application still performs execution and must return the result to the model.

The operation's implementation must already exist and be connected to the tool name. A tool description alone neither creates that implementation nor executes it. The teaching text convention and provider-native function calling are ways of representing the same conceptual request, with different mechanics. The exact provider fields remain in the implementation companion. This page completes the input → request → execution connection; the next chapter develops result delivery, evidence use, and limitations.

Sources: [Anthropic · Building effective agents](#); [Anthropic · Tool definitions and model context](#).

2.8 Choosing a call, an answer, or a clarification

Slide 18.

Appropriate tool use depends on the information the task needs, the available tools, and the required inputs.

EXAMPLES · EXPECTED RESPONSES TO DIFFERENT TASKS:

"Explain precipitation probability." → answer the concept question.

"Retrieve tomorrow's forecast for Seoul." → request Forecast.

"Retrieve tomorrow's forecast here." → ask for the city.

A tool-use model can learn these distinctions. Descriptions and instructions guide its selection; they do not guarantee a correct choice.

Concept check: What must happen between supplying a tool description and obtaining a forecast?

Answer: The model must generate a suitable request. The application must recognize it, execute the corresponding tool, and return the result.

Sources: [Anthropic · Tool definitions and model context](#); [Schick et al. \(2023\) · Toolformer](#).

3. Using tool results in an answer

Slide 19.

3.1 Motivation: returning the result to the model

Slide 20.

A completed lookup is useful only when its result reaches the model.

If the application retrieves an 80% rain probability but omits it from the next input, the model still lacks that forecast.

Purpose: explain how a returned result supports an answer and which claims the result can justify.

Sources: [Anthropic · Building effective agents](#).

3.2 Adding a tool result to the next model input

Slide 21.

The application adds the request and tool result to the original input.

EXAMPLE · INPUT FOR THE NEXT MODEL RESPONSE:

RETAINED INPUT (RELEVANT EXCERPT)

Instruction: Request Forecast before giving weather advice.

Tool: Forecast returns weather for a given city and day.

User: Will I need an umbrella in Seoul tomorrow?

ADDED AFTER EXECUTION

Earlier request: Forecast; city: Seoul; day: tomorrow.

Tool result: "Seoul, tomorrow: 80% probability of rain."

The model now generates from an input that includes the forecast. The input changes; the model parameters do not.

The earlier model input contained the system instructions, descriptions, and user question shown in Chapter 2. The application now includes the preceding request and its returned observation as additional context, while retaining the relevant instructions and history. The example displays the content relevant to the conceptual comparison rather than complete provider serialization. The next page shows an answer generated from this evidence. This is the same autoregressive generation mechanism with an expanded input, not a parameter update.

The retained input shown on the slide is an excerpt. The application also retains the rest of the system prompt, including the Calculator definition, request format, and missing-city rule. These are omitted from the display to focus on the newly appended request and result.

Sources: [Schick et al. \(2023\) · Toolformer](#); [Anthropic · Building effective agents](#).

3.3 Example: advice supported by the forecast

Slide 22.

An evidence-grounded answer uses relevant tool results to support its claims.

EXAMPLE · ILLUSTRATIVE FORECAST AND ANSWERS:

Instruction: Base the recommendation on the retrieved forecast.

User: Will I need an umbrella in Seoul tomorrow?

Tool result: "Seoul, tomorrow: 80% probability of rain."

Supported answer: Rain is likely; carrying an umbrella is advisable.

Unsupported answer: It will definitely rain tomorrow.

The result supports a precautionary recommendation. It does not support certainty about tomorrow's weather.

The observation and answer are course-authored. No weather lookup or live model evaluation was performed. The example makes the inference explicit: predicted rain risk supports carrying an umbrella under ordinary preferences. Grounding in a forecast does not convert a probabilistic forecast into an observed future outcome.

Sources: [Anthropic · Building effective agents](#).

3.4 Checking the relevance and reliability of a result

Slide 23.

A returned result supports an answer only if it matches the task and comes from an appropriate source.

EXAMPLES · RESULTS THAT DO NOT SUPPORT THE ANSWER:

Location: a Busan forecast does not answer a question about Seoul.

Date: today's forecast does not answer a question about tomorrow.

Source: an old web post does not establish the latest forecast.

Successful execution means the tool returned data. The model must still check what those data establish for the user's question.

These are epistemic checks, not JSON validation. A perfectly well-formed request and successful service response can still fail to support the intended conclusion. The location, date, and uncertainty examples are course-authored. Tool outputs should not be treated as automatically authoritative.

Sources: [Anthropic · Building effective agents](#); [Anthropic · Writing effective tools](#).

3.5 A failed lookup and an unknown forecast

Slide 24.

A service error explains why the lookup failed; it provides no information about the weather.

EXAMPLE · AN ERROR RESULT AND ITS MEANING:

User: Will I need an umbrella in Seoul tomorrow?

Tool result: "Forecast service unavailable."

Supported answer: I could not retrieve the forecast.

Unsupported answer: No rain is expected.

The model should report the missing evidence or seek another source. It cannot treat retrieval failure as a forecast.

Concept check: Why is "the lookup failed" different from "no rain is forecast"?

Answer: The first reports an unsuccessful information-gathering operation. The second is a weather claim that requires forecast evidence.

Sources: [Anthropic · Building effective agents](#).

4. Tool sequences and action outcomes

Slide 25.

4.1 Motivation: tasks that require a later action

Slide 26.

A request to reserve an available item requires both a stock lookup and a reservation.

The stock result determines whether a reservation should be attempted and which item it should name.

Purpose: explain how one tool result determines a later call and how to verify the action's outcome.

Sources: [Ng · Agentic AI, Module 3](#); [Anthropic · Building effective agents](#).

4.2 A later call that depends on an earlier result

Slide 27.

Tool calls are dependent when an earlier result determines a later call's inputs or whether that call should occur.

EXAMPLE · A LOOKUP RESULT USED IN A RESERVATION:

User: Find a black coat in size M; reserve one if available.

First call: Stock lookup; color: black; size: M.

Tool result: item C17; two available.

Later call: Reservation; item: C17; quantity: one.

The lookup supplies C17 and establishes availability. Without that result, the model cannot justify this reservation request.

The inventory lookup and reservation are hypothetical. This example establishes observation-dependent action composition without introducing ReAct's explicit reasoning protocol, which remains the subject of Week 4.

Sources: [Ng · Agentic AI, Module 3](#); [Anthropic · Building effective agents](#).

4.3 Reading inventory and changing inventory

Slide 28.

External state is information maintained outside the model, such as inventory. A lookup reads that state; a reservation changes it.

EXAMPLE · READ AND WRITE OPERATIONS ON INVENTORY:

Stock lookup: inventory is two before and after the lookup.

Returned information: item C17 has two available coats.

Reservation of one coat: inventory changes from two to one.

Returned outcome: reservation R204 is confirmed.

Reading available stock does not reserve it. Only a successful reservation changes the inventory in this example.

The inventory example contrasts lookup with reservation. Lookup supplies information about availability; reservation requests a change in availability. Observing a state does not reserve it. The example abstracts from provider APIs and database mechanisms.

Sources: [Ng · Agentic AI, Module 3](#); [Anthropic · Building effective agents](#).

4.4 Example: a reservation from request to confirmation

Slide 29.

The final reservation claim must be based on the reservation result.

EXAMPLE · ILLUSTRATIVE CALLS, RESULTS, AND ANSWER:

User: Check a black coat in M; reserve one if available.

Tools: Stock lookup reads availability; Reservation reserves an item.

Lookup request: black, M.

Lookup result: item C17; two available.

Reservation request: item C17; quantity one.

Reservation result: confirmed, R204; one remains available.

Model answer: One coat is reserved; confirmation R204.

The stock result supports the decision to request a reservation. The reservation result supports the claim that it succeeded.

The tools are independent capabilities connected by the model's requests. The stock lookup identifies the item and supplies availability evidence. The reservation request uses that observed item identifier; reservation success and the remaining inventory are externally reported outcomes. The user request, observations, state change, and final answer are hypothetical. A later action can fail even after a successful lookup, as the following page explains.

Sources: [Ng · Agentic AI, Module 3](#); [Anthropic · Building effective agents](#).

4.5 Available stock and a failed reservation

Slide 30.

Stock can change after a lookup, so the lookup result cannot guarantee a later reservation.

EXAMPLE · A STATE CHANGE BETWEEN LOOKUP AND ACTION:

Stock lookup: one coat is available.

Before reservation: another customer reserves that coat.

Reservation result: rejected; no stock remains.

Supported answer: The coat could not be reserved.

The action's result determines whether it succeeded. Earlier availability supports an attempt, not a claim of completion.

The state-change example is course-authored. A lost response can also leave completion uncertain: absence of confirmation does not establish absence of an effect. Execution-time coordination, retry policies, and idempotency mechanisms are deferred to the optional implementation reference.

Concept check: Does observing available stock establish that a reservation succeeded?

Answer: No. Availability is an earlier observation; reservation is a separate action whose outcome must be established.

Sources: [Anthropic · Building effective agents](#); [AWS Builders' Library · Idempotent APIs](#).

5. Training models to use tools

Slide 31.

5.1 Motivation: learning to generate useful calls

Slide 32.

A tool description identifies an available operation. The model still needs the ability to generate a useful call and use its result.

Training examples can teach when to request a calculator and how to continue after its answer.

Purpose: distinguish prompting from parameter training and examine the training data used by Toolformer and ToolLLM.

Earlier sections explain behavior during inference. This section changes the explanatory level to the acquisition of capability through parameter updates. Toolformer and ToolLLM provide specific research examples of training-data construction; neither paper is a claim about the undisclosed training process of every deployed model. The comparison concerns supplying current input versus learning from examples through a training objective.

Sources: [Schick et al. \(2023\) · Toolformer](#); [Qin et al. \(2023\) · ToolLLM](#).

5.2 Prompting a model and training a model

Slide 33.

Prompting changes the model's input. Training changes its parameters so useful requests and answers become more likely.

EXAMPLES · INPUT CHANGES AND PARAMETER UPDATES:

Prompting: supply "Calculator evaluates an arithmetic expression."

In-context learning: also supply examples of when and how to call it.

Fine-tuning: update parameters using tool-use training examples.

In-context learning uses the current context with fixed parameters. Adding or rewriting a tool description does not fine-tune the model.

In-context learning uses task descriptions or examples at inference without gradient updates.

Fine-tuning changes model parameters. A prompt may elicit an existing generalization ability, but it does not itself perform parameter learning. Toolformer and ToolLLM are research examples of learning tool use, not claims about every deployed model's undisclosed training recipe.

Sources: [Brown et al. \(2020\)](#) · Few-shot learning; [Schick et al. \(2023\)](#) · Toolformer.

5.3 Toolformer: building training text with tool calls

Slide 34.

Toolformer trains a model on text containing useful tool calls and their results.

A result is useful if it improves prediction of the following original text. Fine-tuning then teaches the model to generate calls in such contexts.

Figure description: Use demonstrations to propose call insertions → execute the candidate tools → retain calls whose results reduce prediction loss → fine-tune on the augmented text.

Prediction loss is the weighted negative log-probability of subsequent original-text tokens under the model. The filter compares the result-bearing insertion with the better of two baselines: no insertion and an insertion without the result. A small set of demonstrations supports proposal generation; the approach should not be described as requiring no initial examples. The subsequent text is the training target, not a premise already supplied to the model at the position being scored. The example 437 is a known reference answer in the corpus; the filter asks whether a returned tool result improves prediction of that continuation.

Sources: [Schick et al. \(2023\)](#) · Toolformer.

5.4 Example: selecting a useful calculator call

Slide 35.

Toolformer keeps a call when its result helps the model predict the original training text. Prediction loss is lower when that text is more probable.

EXAMPLE · TOOLFORMER FILTER (SCHEMATIC):

Original text: "Nineteen times twenty-three is 437."

Continuation to predict in every condition: 437.

A. Text prefix only → loss L_A .

B. Text prefix and a request to calculate 19×23 → loss L_B .

C. Text prefix, the request, and returned result 437 → loss L_C .

Keep the call if $\min(L_A, L_B) - L_C \geq \tau$.

τ is the required loss reduction. The returned result must help more than either no call or a call without its result.

This is a teaching reconstruction of the filtering setup, not a measured model run or the paper's literal serialization. At the position being evaluated, the original continuation 437 is a target whose likelihood is scored. It is not already available in the no-call prefix. Condition C makes that value available through a tool result. The paper uses weighted losses on subsequent original-text tokens, so the target is not limited to a single numeric token.

A, B, and C all score the same reference continuation. L denotes weighted negative log-probability over the following original-text tokens; the numeric answer is a short illustration. The displayed ordering is schematic: in the original filtering experiment the tool representation is prepended before the text prefix. Training later inserts retained calls into the original text. This distinction does not change the comparison of the three conditions.

Sources: [Schick et al. \(2023\)](#) · Toolformer.

5.5 ToolLLM: training on complete task solutions

Slide 36.

ToolLLM trains ToolLLaMA on ToolBench solution trajectories: recorded sequences of reasoning, calls, results, and answers for a task.

EXAMPLE · ILLUSTRATIVE TRAINING TRAJECTORY (EXCERPT):

Task: Compare the same coat in stores A and B.

Call A's price tool → result: 120 dollars.

Call B's price tool → result: 100 dollars.

Target answer: B is cheaper by 20 dollars.

During training, earlier results are inputs for predicting later calls and the answer. A complete solution teaches tool use across several steps.

This is a course-authored excerpt illustrating task-level trajectory supervision, not a literal ToolBench run. ToolBench includes task instructions and solution paths collected using an external model and tools. The simplified trajectory shows model-produced requests and an answer as training targets; tool observations provide context. A complete source trajectory can also contain reasoning. Fine-tuning updates model parameters so appropriate requests and answers become more likely under their preceding context. At inference, the learned model must generate a path for the current task rather than retrieve this teaching example verbatim. This connects the training chapter to the conditional-generation mechanism in Chapter 2.

The visible trajectory omits reasoning to keep the supervision relation clear. ToolBench solution paths may include reasoning as well as requests and results; model-produced reasoning, calls, and final answers are supervised targets, while tool results provide context.

Sources: [Qin et al. \(2023\)](#) · ToolLLM; [Schick et al. \(2023\)](#) · Toolformer.

5.6 ToolLLM: finding successful training examples

Slide 37.

ToolLLM uses depth-first search-based decision trees (DFSDT) to find successful solution paths, including alternatives to failed attempts.

EXAMPLE · FINDING A USABLE TRAJECTORY:

Task: Compare a coat's price in stores A and B.

First path: obtain A's price; B rejects an unknown product ID.

Alternative path: search B for the coat, then request its price.

Retained solution: obtain both prices and answer the comparison.

Search produces successful examples for training. Training on those examples does not itself run the search procedure.

Course-authored illustration, not a measured ToolBench run. DFSDT is a search procedure used in solution-path construction: an unsuccessful branch can be abandoned and another path explored. Training on selected trajectories is distinct from running that search procedure at inference. The trained model is not assumed to execute DFSDT automatically.

Concept check: How do Toolformer and ToolLLM differ in the training examples they construct?

Answer: Toolformer filters API call–result insertions using subsequent-token loss. ToolLLM generates task-level solution trajectories through path exploration.

Sources: [Qin et al. \(2023\)](#) · ToolLLM; [Schick et al. \(2023\)](#) · Toolformer.

6. Evaluating tool use

Slide 38.

6.1 Motivation: checking more than the final answer

Slide 39.

A fluent answer can claim success even when a tool call failed.

“Your coat is reserved” is correct only if the reservation result confirms it. The sentence alone does not establish completion.

Purpose: evaluate the selected calls, the evidence used, and the outcome of the requested action.

Sources: [Anthropic](#) · [Writing effective tools](#).

6.2 Evaluation of calls, evidence, and outcomes

Slide 40.

Call selection: the requested operation and its inputs must fit the task. For a Seoul forecast, the call must name Seoul and the requested day.

Evidence use: the answer must follow from the returned result. An 80% rain probability supports advice, not certainty.

Action outcome: a completion claim must agree with the action's result. Available stock alone does not prove a reservation.

Week 3 explains tool use as a capability involving learned generation and external feedback. Week 4 develops ReAct as an explicit method for coordinating reasoning and action over a sequence of decisions. These criteria operationalize the same tool-use cycle: task and descriptions, generated request, external execution, returned result, and the next response. Tool-use training affects generation, but evaluation still requires checking actual requests and outcomes. The Chapter 4 coat example and the lab's clock-to-file exercise differ in domain, not in their result dependency.

6.3 Lab: observing the tool-use cycle

Slide 41.

Clock lookup and saving: inspect the supplied tool description, the generated call, and the result returned to the model.

Dependent calls: save that returned time in a file and compare the actual file content with the clock result.

Tool descriptions and policies: revise the inputs to improve selection and behavior. The model parameters remain fixed.

The implementation companion provides provider fields, input schemas, result identifiers, and control settings. The lab shows description delivery through a system message and through

separately supplied tools. The core clock-to-file task verifies that a later request uses a returned observation. Description and policy editing change inference-time input; this lab does not update model parameters.

Concept check: What evidence shows that a returned timestamp was actually saved?

Answer: A later write request must use the clock result, and the saved file content must match that result. The model's final claim alone is insufficient.

6.4 Readings on tool use and its training

Slide 42.

Schick et al., Toolformer: learning useful tool calls from augmented text. Qin et al., ToolLLM: training on task solution trajectories.

Ng, Agentic AI, Module 3; Anthropic, Building Effective Agents and Writing Effective Tools: tool design and external execution.

Brown et al.: in-context learning. Yao et al., ReAct (Week 4): coordinating reasoning and actions. Source links are in the notes.

Sources: [Brown et al. \(2020\) · Few-shot learning](#); [Schick et al. \(2023\) · Toolformer](#); [Qin et al. \(2023\) · ToolLLM](#); [Ng · Agentic AI, Module 3](#); [Anthropic · Building effective agents](#); [Anthropic · Writing effective tools](#); [Yao et al. \(2022/2023\) · ReAct](#).

Optional implementation reference

The [implementation companion](#) covers schemas, call identifiers, and execution details for the later lab. The concept briefing is complete without these interface details.